

Министерство образования Республики Беларусь

Учреждение образования

«Белорусский государственный университет информатики
и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

Лабораторная работа №3
«Взаимодействие и синхронизация процессов»

Выполнил:
Студент группы 350501
Рутковский В.К.

Проверил:
Поденок Л.П.

Минск 2025

1 ПОСТАНОВКА ЗАДАЧИ

Управление дочерними процессами и упорядочение вывода в `stdout` от них, используя сигналы `SIGUSR1` и `SIGUSR2`.

2 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

2.1 Описание работы программы

Программа реализует управление дочерними процессами с помощью родительского процесса, обрабатывающего команды пользователя. Родительский процесс создает и удаляет дочерние процессы, а также управляет их жизненным циклом с использованием сигналов и таймеров.

После запуска родительский процесс переходит в цикл обработки пользовательского ввода. Команда `+` инициирует создание нового дочернего процесса, о чём родитель сообщает в консоль. Команда `-` удаляет последний из созданных дочерних процессов и выводит количество оставшихся. При вводе `l` программа отображает список всех активных дочерних процессов и самого родителя. Символ `k` приводит к завершению всех дочерних процессов с соответствующим уведомлением. Команда `q` завершает работу родительского процесса после удаления всех дочерних.

Каждый дочерний процесс запускается в отдельной ветке и выполняет независимый цикл обработки данных. Внешний цикл дочернего процесса включает таймер с задержкой в 2 секунды, реализованный через `nanosleep()`. Внутри внешнего цикла выполняется бесконечный внутренний цикл, в котором структура из двух целых чисел последовательно принимает значения $\{0, 0\}$ и $\{1, 1\}$ с чередованием. Поскольку операция записи не является атомарной, при срабатывании таймера состояние структуры может быть непредсказуемым — возможны любые комбинации из 0 и 1.

Каждое срабатывание таймера запускает проверку содержимого структуры, фиксируются статистические данные по различным комбинациям значений. По достижении заданного количества повторений (например, 101 внешнего цикла), дочерний процесс выводит на экран информацию о себе: `PPID`, `PID` и статистику — количество каждой из 4 возможных комбинаций значений в структуре, зафиксированных в момент таймера.

Программа демонстрирует взаимодействие процессов через системные вызовы `fork()`, `kill()`, `waitpid()` и использование сигналов и таймеров для асинхронного анализа данных в параллельных процессах.

2.2 Описание основных функций

Программа состоит из нескольких подпрограмм, каждая из которых реализует отдельную часть логики взаимодействия между родительским и дочерними процессами. К основным модулям относятся:

- Создание дочернего процесса;
- Завершение всех процессов или только последнего;
- Вывод информации о родительском и списке дочерних процессов;
- Обработка команд, введенных пользователем (+, -, l, k, q) и выполнение соответствующих действий.
- Цикл дочернего процесса, в котором анализируются значения в структуре, считается статистика, которая выводится по окончании таймера.

Функция `set_noncanonical_mode`

Функция переводит терминал в неканонический режим, отключая буферизацию ввода и отображение символов при наборе. Для этого она получает текущие настройки терминала с помощью `tcgetattr`, сохраняет их в структуре `oldt`, затем изменяет флаг `c_lflag`, убирая `ICANON` (канонический режим) и `ECHO` (отображение ввода), и применяет новые настройки через `tcsetattr`.

Функция `restore_terminal_mode`

Функция восстанавливает нормальный (канонический) режим терминала, используя сохранённую ранее структуру `oldt`. Она просто передаёт сохранённые настройки обратно в терминал с помощью `tcsetattr`.

Функция `main`

Главная функция программы инициализирует работу с терминалом, запускает цикл обработки ввода и управляет созданием и завершением дочерних процессов. Сначала она переводит терминал в неканонический режим, устанавливает обработчики сигналов для родителя, и выводит инструкции пользователю. Затем в цикле считывает символы с клавиатуры и выполняет соответствующие действия: + — создаёт дочерний процесс через `fork()`, - — завершает последний созданный дочерний процесс, l — выводит список текущих дочерних процессов, k — завершает всех детей, q — завершает программу и всех детей. После выхода из цикла восстанавливает стандартный режим терминала.

Функция `parent_sigusr1_handler`

Эта функция является обработчиком сигнала `SIGUSR1`, получаемого родительским процессом. Она использует расширенный формат обработки сигналов (`siginfo_t`) и, если информация о процессе-источнике доступна, отправляет этому процессу ответный сигнал `SIGUSR2` с помощью `kill(info->si_pid, SIGUSR2)`. Это используется для взаимодействия между дочерним и родительским процессами.

Функция `setup_parent_signals`

Функция настраивает обработку сигнала SIGUSR1 для родительского процесса. Она инициализирует структуру `sigaction`, устанавливает поле `sa_sigaction` на функцию `parent_sigusr1_handler`, включает флаг `SA_SIGINFO` для получения расширенной информации о сигнале и регистрирует обработчик с помощью `sigaction`.

Функция `kill_all_children`

Функция завершает все активные дочерние процессы. Сначала она проходит по массиву `children`, отправляя каждому процессу сигнал SIGTERM и выводя сообщение о завершении. Затем в цикле с помощью `wait` ожидает завершения всех процессов и удаляет их PID из массива `children`, сдвигая элементы массива влево, чтобы поддерживать его актуальное состояние. В конце счётчик `num_children` обнуляется.

Функция `child_sigusr2_handler`

Это обработчик сигнала SIGUSR2, используемый дочерним процессом. При получении сигнала функция устанавливает глобальную переменную `print_allowed` в значение 1, позволяя дочернему процессу напечатать статистику. Сама переменная используется как флаг синхронизации между родителем и дочерним процессом.

Функция `child_sigalrm_handler`

Обработчик сигнала SIGALRM, срабатывающий по таймеру. Он считывает значения полей `a` и `b` из общей структуры `sharedPair` и увеличивает соответствующий счётчик (`count00`, `count01`, `count10` или `count11`) в зависимости от комбинации значений. Также увеличивается счётчик общего количества циклов `cycle_count`.

Функция `setup_child_signals`

Функция настраивает обработчики сигналов для дочернего процесса. Сначала связывает сигнал SIGALRM с функцией `child_sigalrm_handler`, затем сигнал SIGUSR2 — с `child_sigusr2_handler`, используя структуру `sigaction`.

Функция `setup_timer`

Настраивает периодический таймер для дочернего процесса. С помощью структуры `itimerval` задаёт интервал срабатывания сигнала

SIGALRM и запускает таймер через `setitimer`, используя интервал, заданный макросом `TIMER_INTERVAL_USEC`.

Функция `child_process`

Главная функция, выполняемая в дочернем процессе. Она устанавливает обработчики сигналов и запускает таймер. Затем в бесконечном цикле генерирует последовательность значений `sharedPair.a` и `sharedPair.b` со сном между переключениями (задержка задаётся `SLEEP_NSEC`). После завершения заданного количества циклов (`CYCLES`) процесс отправляет родителю сигнал `SIGUSR1` и ждёт разрешения на вывод (`print_allowed`). После получения сигнала выводит статистику по комбинациям значений и сбрасывает все счётчики.

2.3 Описание запуска и сборки

Для сборки проекта используется утилита `smake` и файл `CMakeLists.txt`, которая автоматически создает `makefile`, с помощью которого производится компиляция исходного кода и создание исполняемых файлов из `child.c` и `parent.c`. Запускать `smake` следует в директории `build` для структуризации проекта (если находитесь в папке `build`: `$smake ../CMakeLists.txt`; далее: `$make`).

Для удаления скомпилированного файла и очистки директории `build` предусмотрена команда `make clean`. Это позволяет поддерживать рабочую среду в чистоте и гарантировать корректность последующих сборок.

3 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

```
Parent: Created child PID=5080 (total: 1)
Parent: Created child PID=5081 (total: 2)
Parent: Terminated child PID=5081
Parent PID=5051
  Child 1: PID=5080
Child [PPID=5051, PID=5080]: {0,0}=34, {0,1}=34, {1,0}=33,
{1,1}=0
```

Рисунок 3.1 – Обработка нажатия (+, +,-, k, l)

4 ВЫВОД

В ходе реализации программы было рассмотрено создание и управление процессами с помощью системных вызовов `fork()` и `exec()`, а также их взаимодействие с родительским и дочерними процессами. В процессе работы была использована функция `getpid()` для получения

идентификатора текущего процесса и `getppid()` для получения идентификатора родительского процесса. Программа также использует сигнал `SIGALRM` для периодического выполнения операций в дочернем процессе, с обработкой с использованием `sigaction`. Системный вызов `setitimer()` был использован для установки таймера, который вызывает сигнал через определённый интервал времени.